Relatório 5 - Prática: Formulários e UX para Dashboards (II)

José Carlos Seben de Souza Leite

Descrição da atividade

Neste card começamos sobre a funcionalidade do **React Hoock Form** uma biblioteca para formulários como diz o nome, mas com um grande diferencial que é seu impacto no desempenho, diferente do nativo do React que além de ser muito verboso para a criação de um formulário em cada input teremos que adicionar um useState que desencadearia em cada letra digitada uma atualização e feria o site re-renderizaria gerando um desperdício enorme de poder computacional, o **RHF** resolve este problema com seus “componentes não controlados” que geram essa atualização apenas quando o texto escrito for “enviado” gerando um impacto direto no desempenho do site.

Agora uma biblioteca adicional para utilizarmos com o **RHF** é a **Zod** uma biblioteca muito útil para criação de formulários que se aproveita muito do “buff” de desempenho que temos usando o **RHF** e facilitando muito a criação desses formulários, pois tem funções pre prontas para a validação de dados de entrada como verificar o tipo de uma entrada de é um e-mail se esta realmente em formato de um como algo parecido com jose@gmail.com, verificar entrada de senhas, por exemplo, com a função .**refine** que tem um exemplo a baixo que esta sendo utilizado para a comparação de senha na tela de registro onde o usuário precisa digitar a senha e depois digitá-la novamente para a confirmar.

Neste exemplo abaixo é um dos usos possíveis do **Zod** onde estamos utilizando para a validação de dados de entrada e comparação de senhas.

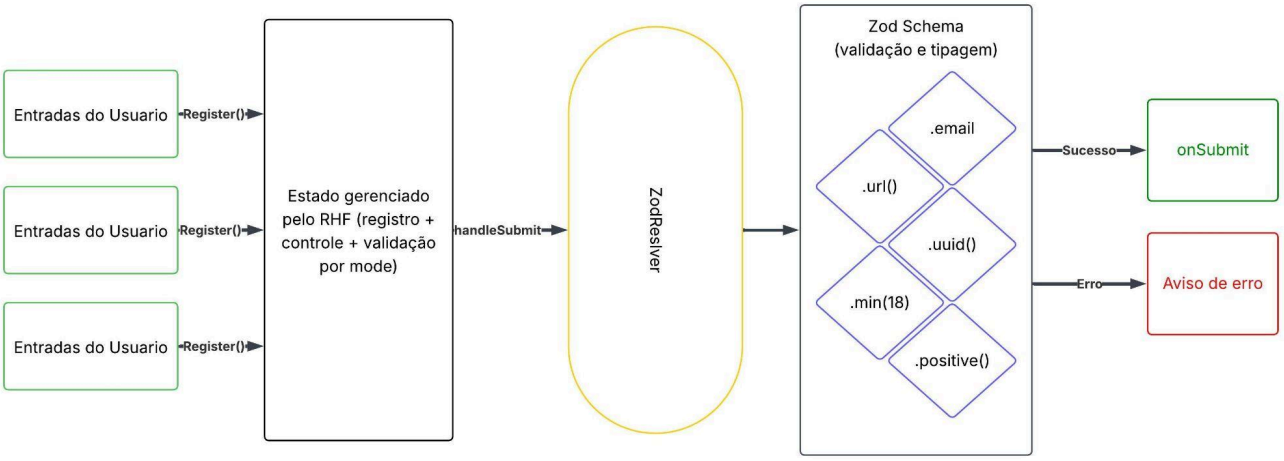
```
1  export const signUpSchema = z
2    .object({
3      name: z.string().min(1, "Nome obrigatório"),
4      email: z.string().min(1, "E-mail obrigatório").email("E-mail inválido"),
5      password: z.string().min(6, "Mínimo de 6 caracteres"),
6      confirmPassword: z.string().min(1, "Confirmação obrigatória"),
7    })
8    .refine((data) => data.password === data.confirmPassword, {
9      message: "As senhas não coincidem",
10     path: ["confirmPassword"],
11   })
```

A verificação de senhas como no exemplo a cima para o cadastro de usuário onde exigimos que o usuário preencha os dados necessários já atribuindo esses valores, assim conseguimos utilizar um dos pontos que citei anteriormente sobre o Zod que é a verificação de, por exemplo, se o e-mail informado esta no formato correto de email e neste caso como é o formulário de criação se conta pedimos a senha e a confirmação da mesma, para isso o Zod recebe as duas entradas e as compara, para fazer essa validação das senhas utilizamos o metodo **.refine()** do zod esta função avalia o objeto de dados na sua totalidade, comparando **data.password** e **data.confirmPassword** antes da submissão, caso as senhas não estejam iguais dispara a propriedade **path** e ele dispara um aviso para o usuário que as senhas não estão iguais.

```
1  export const surveySchema = z.object({
2    id: z.string().uuid(),
3    readingDate: z.date({
4      error: "A data de leitura é obrigatória.",
5    }),
6    theme: z.enum(["Tecnologia", "Economia", "Educação", "Saúde"] as const, {
7      error: "Selecione um tema.",
8    }),
9    frequency: z.enum(["Diariamente", "Semanalmente", "Raramente"] as const, {
10     error: "Selecione uma frequência.",
11   }),
12 })
13
14 export type SurveyResponse = z.infer<typeof surveySchema>
15 export const surveyInputSchema = surveySchema.omit({ id: true })
16 export type SurveyInput = z.infer<typeof surveyInputSchema>
17
```

Outro lugar onde fizemos uso do Zod foi na validação dos dados de entrada no nosso formulário de respostas, basicamente utilizamos métodos como **.string()**, **.uuid(), .date(),** para fazer a validação dos tipos automático do Zod ou **.enum()** para restringir as opções a uma lista predefinida de valores aceitos (como os temas de leitura e a frequência), já configurando as mensagens de erro personalizadas caso o usuário deixe o campo em branco. Para otimizar o código e evitar repetições, aplicamos também o método **.omit()** para reaproveitar este esquema na criação do formulário, removendo apenas a exigência do campo id gerado automaticamente pelo sistema. Por fim, utilizamos o **z.infer** para extrair as tipagens do TypeScript diretamente dessas regras, mantendo a validação e os tipos do projeto sempre sincronizados.

Uma representação visual da funcionalidade do React Hook Form em conjunto com o Zod é:

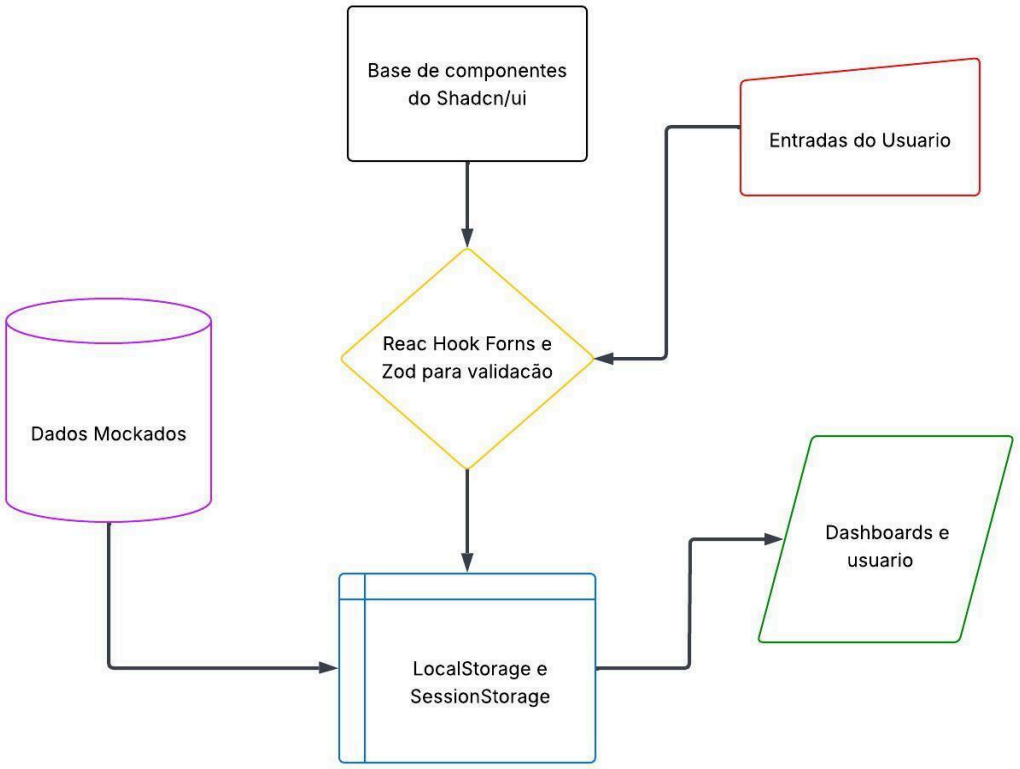


Dando continuidade ao card os vídeos anexados ao card como base ensinam e mostram na prática o desenvolvimento de telas utilizando de base os componentes prontos presentes no shadcn então teoricamente não tem muito para elaborarmos assim partindo para o desenvolvimento das telas pedidas no card que são uma tela de home, sig in/ sig out, uma

tela de dashboards e formulários com o que nos quais nos pudéssemos “alimentar” estes dashboards.

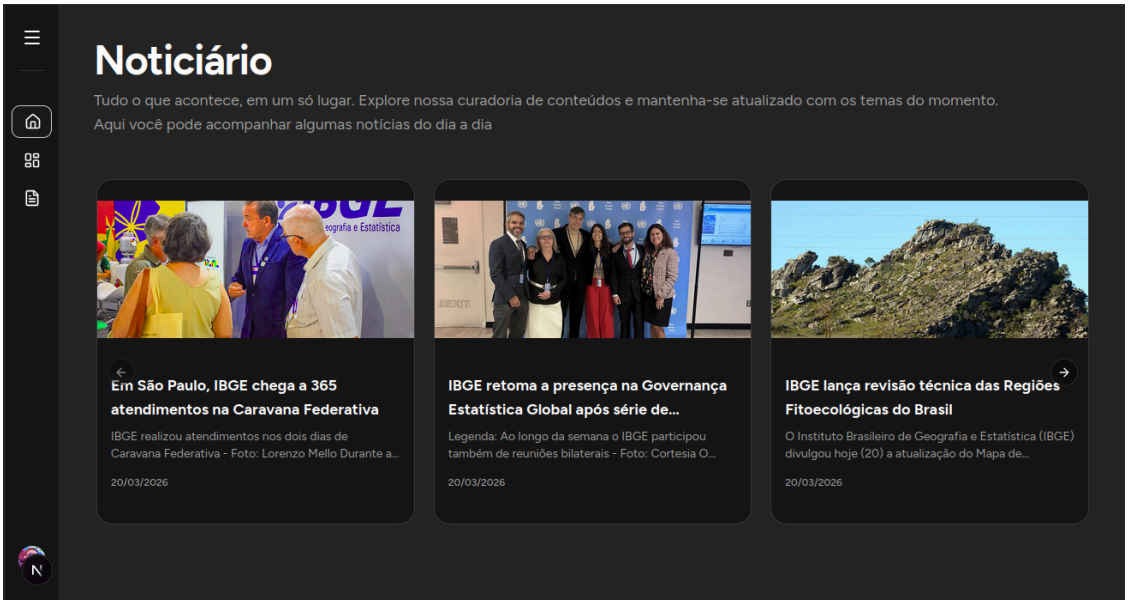
Como isso foi feito, utilizando basicamente os componentes disponíveis no shadcn como base principal, utilizado react hook form com zod para validações de informações e trabalhando com o local storage como ainda não começamos a trabalhar com integração de banco de dados reais, além de aplicação de boas práticas, e arquitetura trabalhados no decorrer do fast camp.

Basicamente oque foi feito é uma ideia simples que tive de fazer algo como um site de notícias onde na home page vai ter algumas notícias vindas da API do IBGE, na tela dos Dashboards terá alguns gráficos que já serão pré-alimentados com alguns dados Mockados criados em um arquivo no nosso sistema que ao rodar serão adicionados ao nosso local storage funcionando como um backup de arquivos não deixando os gráficos sem dados e a tela de Formulários onde o usuário poderá adicionar mais dados a esses gráficos ao responder e salvar sua resposta os dados serão validados pelo zod se corresponder a todos os parâmetros será armazenado no nosso local storage junto dos dados vindo dos mocks assim alimentando os gráficos.

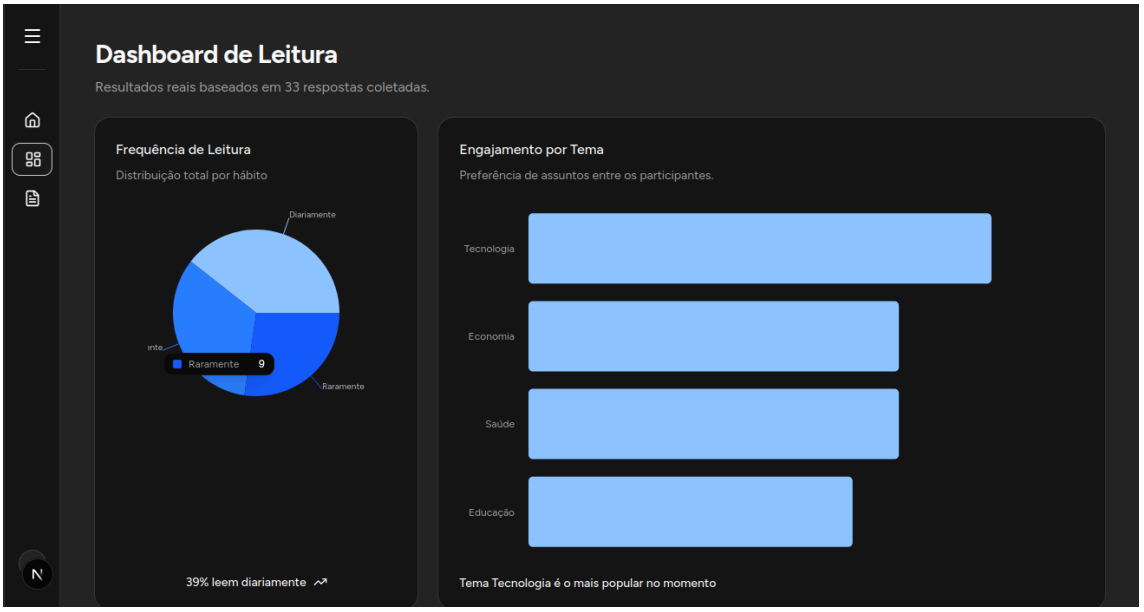


Telas:

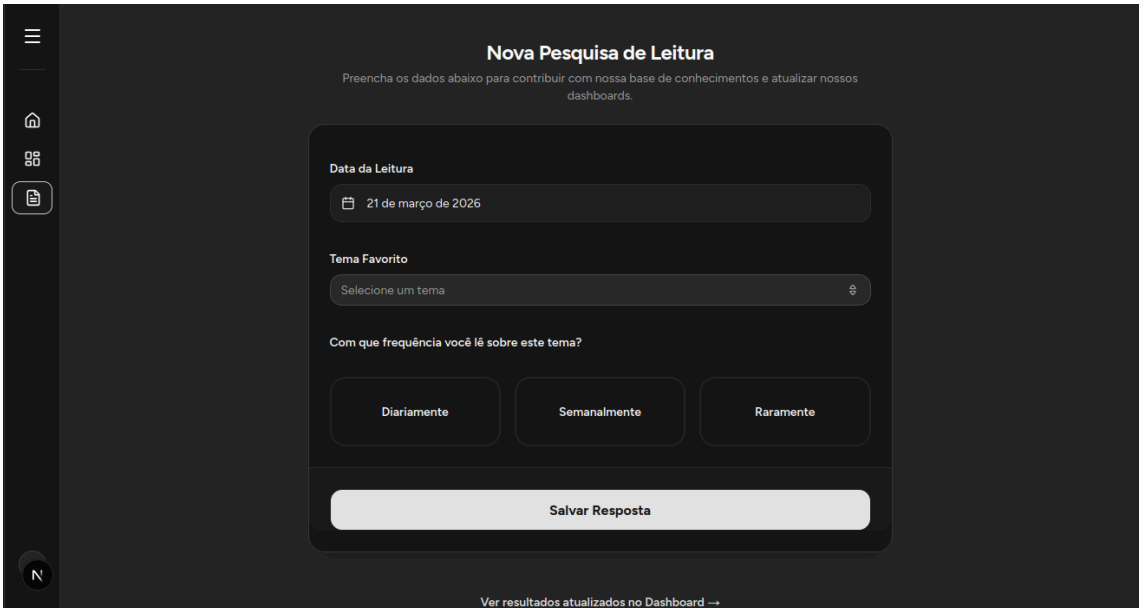
Home Page: A tela inicial do sistema feita utilizando os cards e carrossel do shadcn, o card foi personalizado para armazenar noticias que o sistema receba da API do IBGE utilizada para ter uma alimentação de notícias no sistema, para não fazer uma requisição toda vez que abrir o site ou recarregar a página quando o sistema faz a requisição ele baixa algumas notícias em JSON e as armazena por uma hora no próprio servidor do Next em cache para quando as atualizar alterar os cards sem fazer uma nova requisição, o carrossel está limitado em apenas cinco cards propositalmente como é apenas uma demonstração e está sendo a única tela liberada sem a necessidade de efetuar o login no sistema para quando o usuário abrir o sistema poder visualizar as informações sem precisar criar uma conta logo de início.



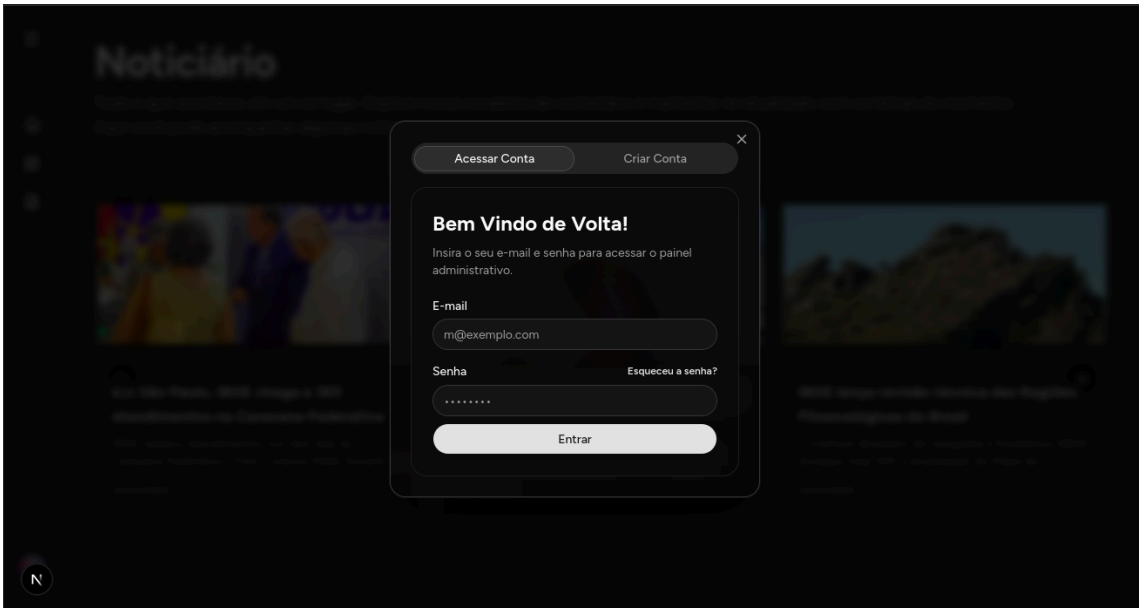
Dashboards: Tela disponível apenas para usuários logados fazendo a verificação de login através do sessionStorage que mantém o login e quando o sistema e fechado e aberto novamente reseta a seção, mas mantém os cadastros armazenados só precisando refazer o login, tela montada com tres gráfico diferente do shadcn que são, Line Chart - Multiple, Bar Chart - Horizontal, e Pie Chart - Label, onde os mesmos são alimentados pelo localStorage alimentado pelos Mocks e respostas do formulário onde cada um pega, uma informação, um com a frequência de leitura, outro com os temas e o último com o mês de leitura.

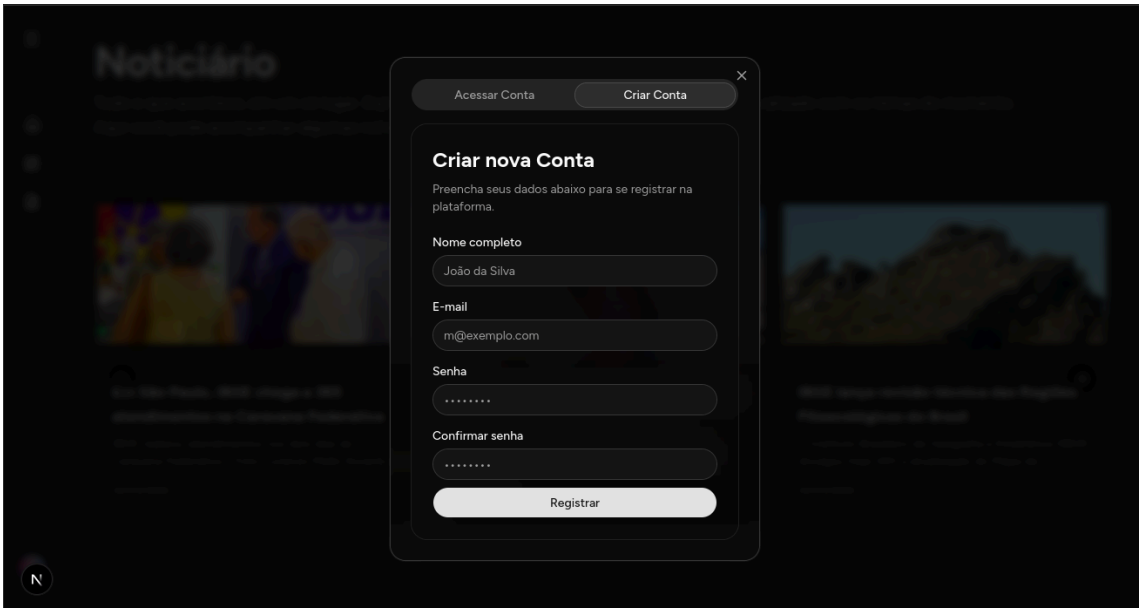


Formulário: Aqui na tela de Formulário acessível apenas para usuários logados com o mesmo tipo de segurança de verificação de usuário logado, é a tela mais simples utilizando alguns componentes para o usuário conseguir da a entrada de dados no sistema em um formulário único onde no mesmo para mandar precisa preencher os três tipos de dados que estamos trabalhando, frequência, tema e data de leitura, utilizando zod para verificação das entradas e campos necessários além de avisos de campos não preenchidos, e suspensão do botão enquanto está efetuando a validação dos dados para impedir múltiplos envios.

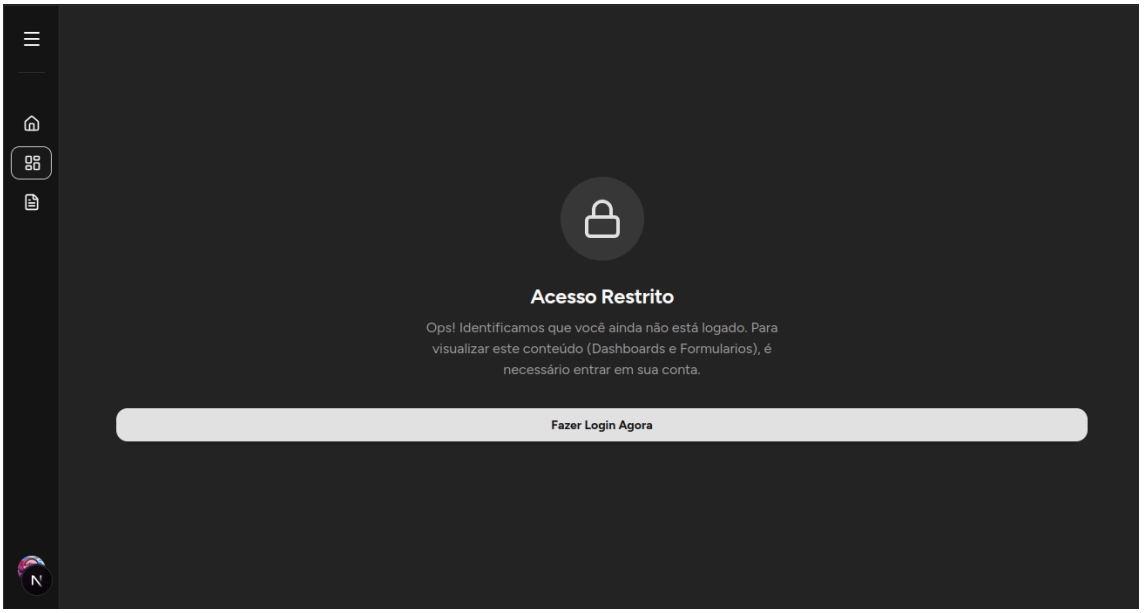


Login: Já para a tela de login foi construído um formulário acessível através da barra lateral quando clica no seu perfil ou ao tentar acessar uma tela com restrição terá um botão que abrira o formulário de login ou cadastro, onde ambos utilizam verificações e avisos com zod para verificar tamanho de senha, formato valido de email, campos preenchidos, se as senhas coincidem, basicamente todas as verificações.





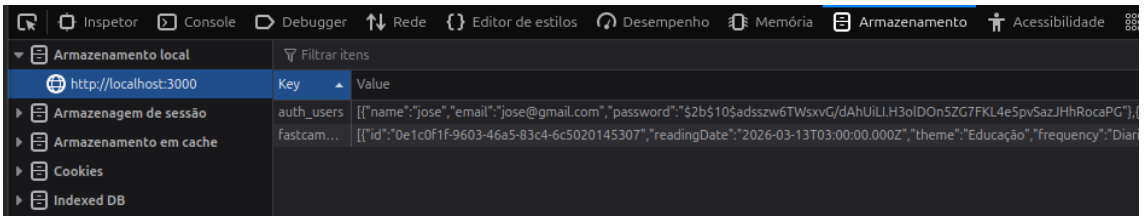
Tela de Bloqueio: Uma tela simples sem muito a falar sobre.



Com nosso sistema de login hipotético que esta servindo apenas para enriquecer um pouco mais o projeto foi sugerido estar utilizando a biblioteca **bcryptjs** para proteção da senha dos usuários.

```
1  async saveUser(user: IUser): Promise<void> {
2      const users = this.getStoredUsers()
3      const hashedPassword = await bcrypt.hash(user.password, 10)
4      users.push({ ...user, password: hashedPassword })
5      localStore.setItem(STORAGE_KEY, JSON.stringify(users))
6    },
```

Como no exemplo a cima da função **saveUser** que recebe um objeto (user) com nome, email, e senha, com essa senha do usuário cria um hash com **bcrypt.hash.(user.password, 10)** e salva esse hash como a senha do nosso novo usuário no sabor banco de dados, assim se alguém acessar nosso local storage ao invés da senha dos usuários exposta se depara com algo como na imagem a baixo:



Dificuldades

Como dificuldades para o desenvolvimento acho que os pontos para citarmos é a personalização dos componentes utilizados do shadcn por falta de costume apenas como, por exemplo, que parte do componente eu vou precisar usar e qual não e outra coisa é respeitar as boas práticas, estar alinhado com as exigências, por exemplo, quando um componente precisa ser dividido ou não ou até quando posso manter algo como esta ou preciso criar um novo Hook, além de estar se acostumando com as diferentes arquiteturas de organização do projeto então tem momentos que me pego pensando “onde devo armazenar isso”, desconfio que são coisas mais ligadas a falta de costume que com a prática vão “desaparecendo”.

Conclusões

Este card até agora apesar do atraso que acabou ocorrendo foi um dos mais divertidos e trabalhosos divertido por eu ter a ideia de oque eu quis construir e ir fazendo isso aos poucos mas trabalhoso ao mesmo tempo, por as dificuldades citadas, mas foi uma boa experiência é gratificante ver o resultado o mais perto possível da ideia inicial.

Referencias

Videos:

- React Hook Form - Complete Tutorial (with Zod)
- Criando UI no React na velocidade da luz! (shadcn/ui)
- Nunca foi tão fácil criar Dashboard com Next JS e Shadcn ui 🔥

Documentações:

https://www.react-hook-form.com/

https://ui.shadcn.com/

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

https://zod.dev/